N G H I Ê N C Ứ U K I Ế N T R Ú C

OpenClaw

Multi-Agent Orchestration Architecture

Tài liệu dành cho Solution Architect

Dự án mã nguồn mở lớn nhất thế giới với hơn 350.000 GitHub stars

Tháng 4/2026

Mục lục

1. Tổng quan về OpenClaw 4
1.1. OpenClaw là gì? 4
1.2. Lịch sử phát triển 4
1.3. Đặc điểm kỹ thuật cơ bản 4
2. Kiến trúc Gateway-Centric 5
2.1. Channel Adapters (Hệ thống con #1) 5
2.2. Session Manager (Hệ thống con #2) 6
2.3. Queue System (Hệ thống con #3) 6
2.4. Agent Runtime (Hệ thống con #4) — “Bộ não” 6
2.5. Control Plane (Hệ thống con #5) 6
3. Ba mô hình Multi-Agent Orchestration 7
3.1. Persistent Agents + Bindings (Tầng routing) 7
3.2. Sub-agents (Tầng runtime) 7
3.3. Lobster Workflow Engine (Tầng workflow) 8
3.4. Kết hợp đồng thời 8
4. Xâu chuỗi ba mô hình trong một tác vụ 9
4.1. Kịch bản chi tiết 9
Bước 1: Tầng Bindings — “Ai xử lý?” 9
Bước 2: Tầng Lobster — “Làm gì, thứ tự nào?” 10
Bước 3: Tầng Sub-agents — “Làm sao cho nhanh hơn?” 10
Bước 4: Quay lại Lobster — Approval và Deploy 10
5. Giao tiếp liên agent và cách ly 11
5.1. Cơ chế giao tiếp 11
5.2. Cách ly per-agent 11
5.3. Team patterns (cộng đồng) 11
6. So sánh với các framework khác 12
7. Năng lực nổi bật 13
7.1. Model-agnostic (BYOM) 13
7.2. Multi-channel 13
7.3. Local-first 13
7.4. Multi-agent 3 tầng 13
7.5. Cấu hình Markdown 13
7.6. Hệ sinh thái 13
8. Rủi ro và hạn chế 14

8.1. Bảo mật 14
8.2. Kiến trúc 14
8.3. Chi phí 14
9. Tài liệu tham khảo 15
9.1. Chính thức 15
9.2. Phân tích cộng đồng 15
9.3. Repo cộng đồng 15

1. Tổng quan về OpenClaw

1.1. OpenClaw là gì?

OpenClaw là một nền tảng AI agent cá nhân mã nguồn mở, cho phép người dùng chạy trợ lý AI tự chủ trên phần cứng riêng và tương tác qua hơn 25 nền tảng nhắn tin. Điểm khác biệt then chốt: trong khi LangGraph hay CrewAI là framework để xây dựng ứng dụng agent, OpenClaw chính là ứng dụng agent — một "hệ điều hành AI" sẵn sàng chạy, với kiến trúc gateway-centric cho phép vận hành nhiều agent độc lập trong cùng một tiến trình.

1.2. Lịch sử phát triển

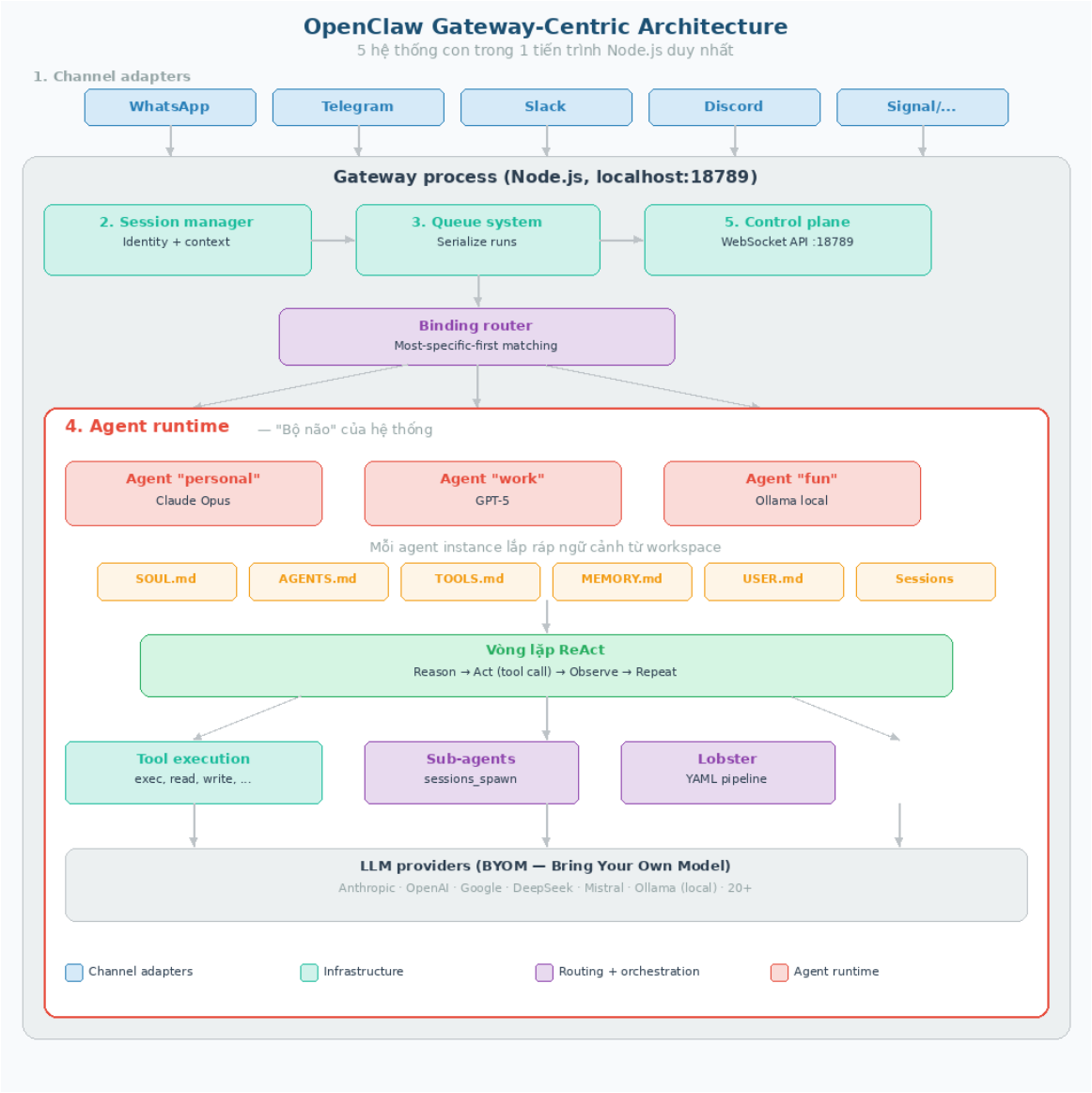
OpenClaw được tạo bởi Peter Steinberger (@steipete), nhà phát triển người Áo. Dự án bắt đầu vào tháng 11/2025 như một WhatsApp Relay đơn giản, trải qua các lần đổi tên: Clawdbot (11/2025) → Moltbot (27/01/2026) → OpenClaw (30/01/2026). Đạt 100.000 stars trong tuần đầu tiên. Ngày 14/02/2026, Steinberger gia nhập OpenAI và thành lập quỹ phi lợi nhuận quản trị dự án.

1.3. Đặc điểm kỹ thuật cơ bản

- **Ngôn ngữ:** TypeScript, chạy trên Node.js 24
- **Giấy phép:** MIT (mã nguồn mở hoàn toàn)
- **LLM providers:** Hơn 20 nhà cung cấp — Anthropic, OpenAI, Google, DeepSeek, Mistral, Ollama
- **Kênh giao tiếp:** 25+ nền tảng — WhatsApp, Telegram, Slack, Discord, Signal, iMessage, Teams
- **Phần cứng tối thiểu:** 2GB RAM, 2 CPU cores (Raspberry Pi)

2. Kiến trúc Gateway-Centric

Toàn bộ hệ thống chạy trong một tiến trình Node.js duy nhất trên cổng localhost:18789. Năm hệ thống con hoạt động đồng thời bên trong, giao tiếp in-process, không cần message broker hay database bên ngoài.



Hình 1: Kiến trúc Gateway-Centric của OpenClaw với 5 hệ thống con

2.1. Channel Adapters (Hệ thống con #1)

Lớp giao tiếp đầu vào/đầu ra. Mỗi nền tảng nhắn tin có adapter riêng (Baileys cho WhatsApp, grammY cho Telegram, discord.js cho Discord, Bolt cho Slack). Chuẩn hóa tin nhắn đến thành định dạng nội bộ thống nhất.

2.2. Session Manager (Hệ thống con #2)

Phân giải danh tính người gửi và ngữ cảnh hội thoại. DM được gộp vào session “main”, chat nhóm nhận session riêng. Lưu dưới dạng file JSONL — local-first, không cần database.

2.3. Queue System (Hệ thống con #3)

Tuần tự hóa các lượt chạy, xử lý ba tình huống: steer (điều hướng lại), followup (xếp hàng chờ), collect (gom nhóm). Đảm bảo không có race condition.

2.4. Agent Runtime (Hệ thống con #4) — “Bộ não”

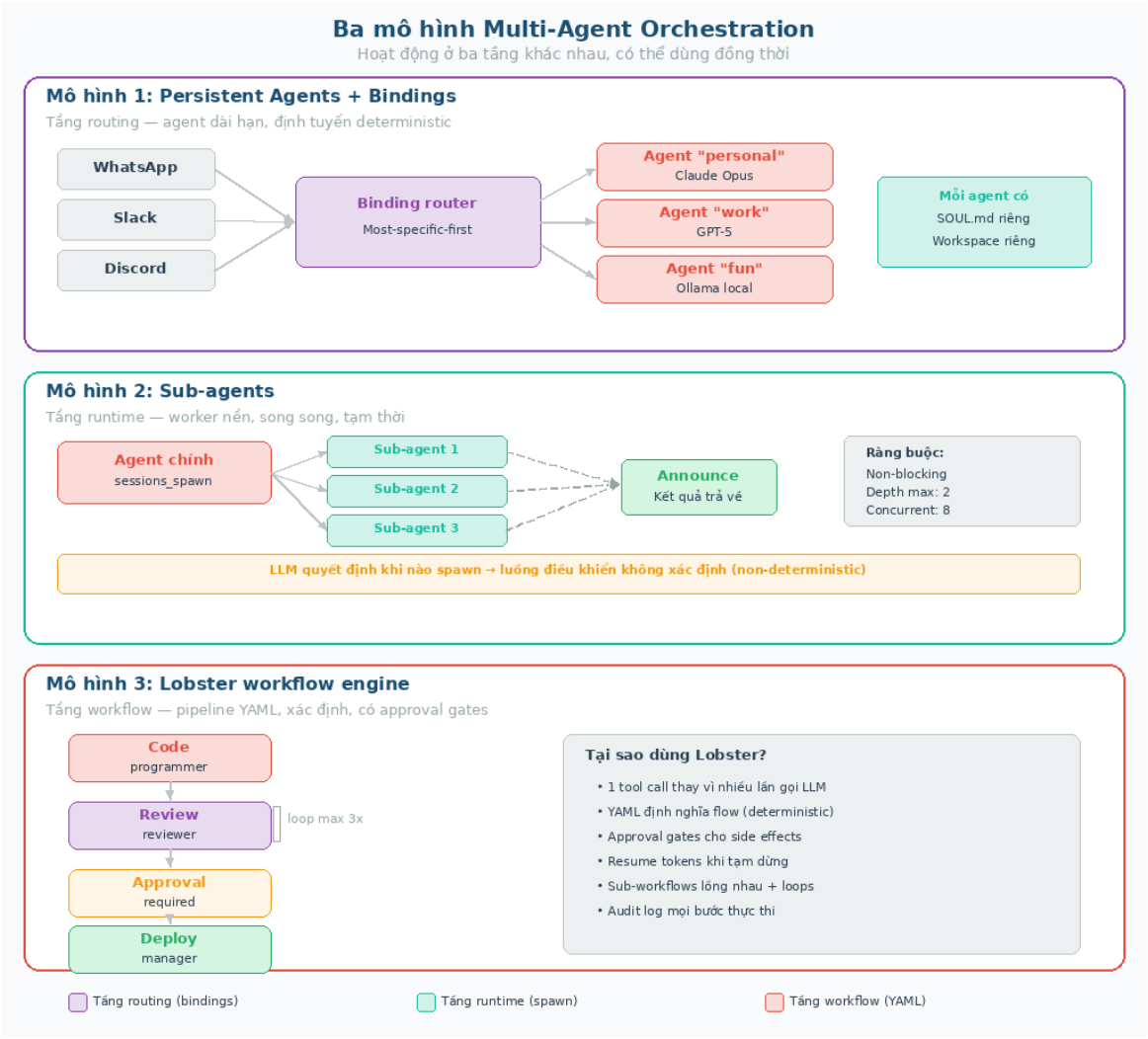
Thành phần lớn nhất, lắp ráp ngữ cảnh từ Markdown files và chạy vòng lặp ReAct. Bao gồm ba lớp: Agent Instances (mỗi agent có workspace riêng), Context Assembly (SOUL.md, AGENTS.md, TOOLS.md, MEMORY.md, USER.md), và Vòng lặp ReAct (Reason → Act → Observe → Repeat). Sub-agents và Lobster là các tool bên trong Runtime mà ReAct loop có thể gọi.

2.5. Control Plane (Hệ thống con #5)

API WebSocket trên cổng 18789, cho phép CLI, ứng dụng macOS, giao diện web kết nối. Điểm điều khiển duy nhất cho toàn hệ thống.

3. Ba mô hình Multi-Agent Orchestration

Ba mô hình hoạt động ở các tầng khác nhau trong kiến trúc và có thể dùng đồng thời, không loại trừ lẫn nhau.



Hình 2: Ba mô hình Multi-Agent Orchestration hoạt động ở ba tầng khác nhau

3.1. Persistent Agents + Bindings (Tầng routing)

Các agent dài hạn được khai báo trong openclaw.json. Hệ thống bindings định tuyến deterministic theo nguyên tắc most-specific-first: peer match → parentPeer → guildId + roles → guildId → teamId → accountId → channel-level → fallback. Nhiều trường khớp trong cùng binding dùng logic AND.

3.2. Sub-agents (Tầng runtime)

Worker nền được sinh ra qua sessions_spawn, chạy non-blocking trong session cách ly, trả kết quả qua cơ chế announce. Ràng buộc: maxSpawnDepth 2, maxConcurrent 8, không có session tools mặc định. Điểm yếu: LLM quyết định khi nào spawn (non-deterministic).

3.3. Lobster Workflow Engine (Tầng workflow)

Engine workflow tích hợp cho phép xây dựng pipeline xác định bằng YAML. Triết lý: dùng LLM cho sáng tạo, dùng YAML cho routing. Tính năng cốt lõi: typed JSON piping, approval gates (human-in-the-loop), resume tokens, sub-workflows với loops, audit log toàn diện. Chạy in-process, một tool call duy nhất thay vì nhiều lần gọi LLM.

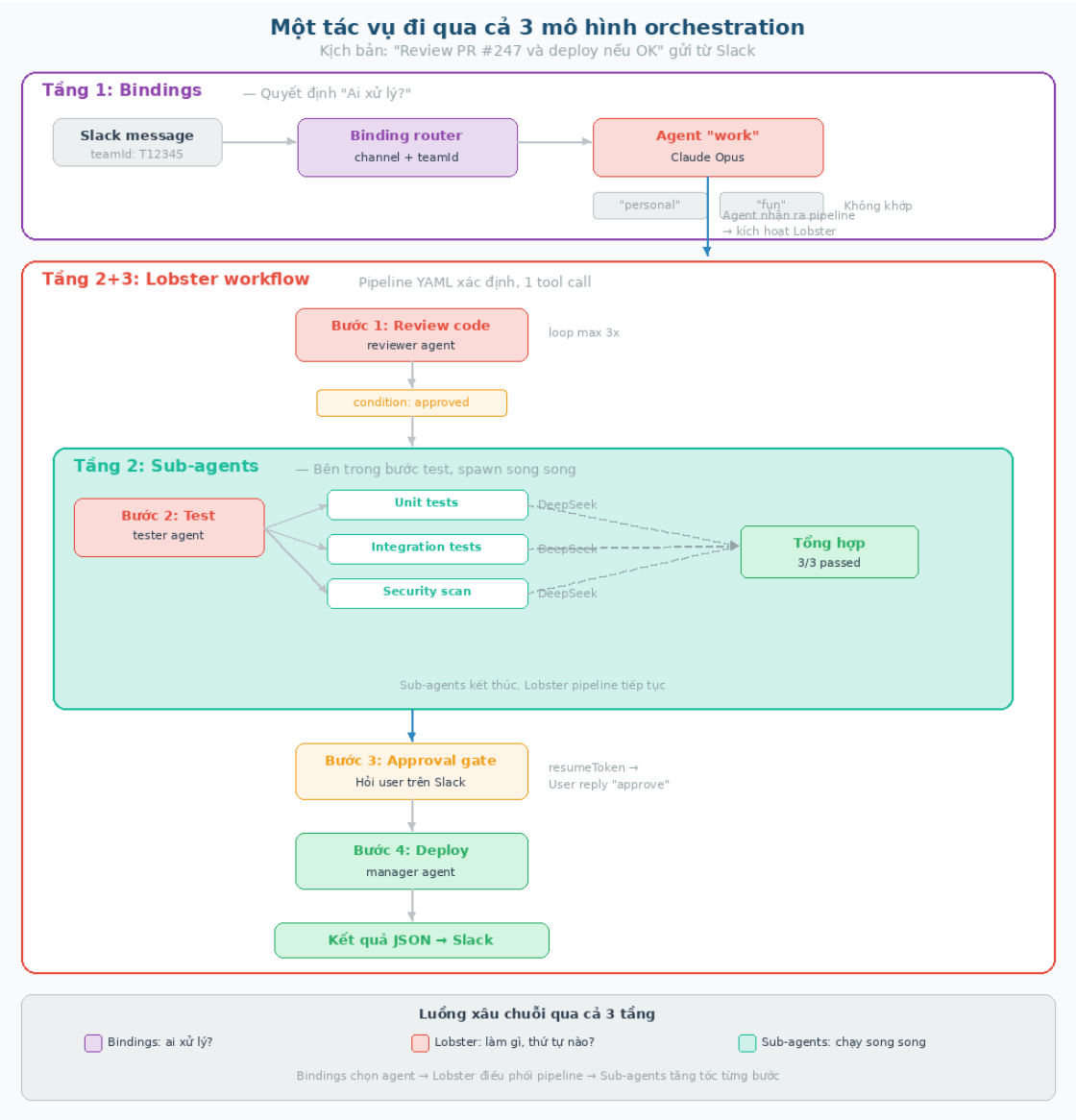
> **Nguyên tắc:** *LLM làm tốt việc viết code, phân tích, sáng tạo. Lobster làm tốt việc tuần tự, routing, retry. Mỗi lần đưa flow control vào prompt đều tạo failure mode.*

3.4. Kết hợp đồng thời

Cách tiếp cận đúng: bắt đầu với một agent → thêm persistent agents khi cần tách kênh/vai trò → bật sub-agents khi cần song song → viết Lobster workflows khi có pipeline lặp lại.

4. Xâu chuỗi ba mô hình trong một tác vụ

Ba mô hình lồng nhau theo thứ bậc trong cùng một tác vụ: Bindings là lớp ngoài cùng, Lobster là khung xương bên trong, Sub-agents được sinh ra bên trong một bước cụ thể của Lobster.



Hình 3: Một tác vụ đi qua cả 3 mô hình — kịch bản "Review PR #247 và deploy nếu OK"

4.1. Kịch bản chi tiết

Bước 1: Tầng Bindings — "Ai xử lý?"

Tin nhắn từ Slack với teamId: T12345. Binding router khớp quy tắc channel + teamId → định tuyến đến Agent "work" (Claude Opus, có quyền Git và CI/CD). Nếu từ WhatsApp, sẽ đến Agent "personal" hoàn toàn khác.

Bước 2: Tầng Lobster — "Làm gì, thứ tự nào?"

Agent "work" nhận ra pipeline đã định nghĩa trong dev-review.lobster, gọi một Lobster tool call. Pipeline: Review (loop 3x) → Test → Approval gate → Deploy.

Bước 3: Tầng Sub-agents — "Làm sao cho nhanh hơn?"

Bên trong bước test, tester agent spawn 3 sub-agents nền (unit tests, integration tests, security scan) chạy trên model rẻ (DeepSeek). Kết quả announce về, tester tổng hợp.

Bước 4: Quay lại Lobster — Approval và Deploy

Pipeline tiếp tục: approval gate gửi resumeToken về Slack hỏi xác nhận. User reply "approve", Lobster resume và deploy.

> **Điểm mấu chốt:** *Bindings trả lời "ai xử lý?", Lobster trả lời "làm gì và thứ tự nào?", Sub-agents trả lời "làm sao cho nhanh hơn?"*

5. Giao tiếp liên agent và cách ly

5.1. Cơ chế giao tiếp

- **agentToAgent:** Nhắn tin trực tiếp giữa agents (tắt mặc định, cần allowlist).
- **sessions_send:** Gửi tin đến bất kỳ session nào, hỗ trợ fire-and-forget và đồng bộ.
- **QMD memory search:** Tìm kiếm bộ nhớ của agent khác qua extraCollections.

5.2. Cách ly per-agent

Mỗi agent được bảo vệ: workspace riêng, auth profiles riêng, session store riêng, tool policies riêng (allow/deny lists), Docker sandbox tùy chọn.

5.3. Team patterns (cộng đồng)

- **Orchestrator:** Agent chính phân tách, ủy quyền, tổng hợp. Phổ biến nhất.
- **Peer-to-peer:** Đàm phán đa bên. Giới hạn ≤5 agents.
- **Hierarchical:** Root → Sub-Orchestrators → Workers. Scale tốt.

6. So sánh với các framework khác

Đặc điểm	OpenClaw	LangGraph	CrewAI	AutoGen
Bản chất	Nền tảng agent	Framework đồ thị	Framework team	Framework hội thoại
Giao diện	25+ kênh chat	API Python	API Python	API Python
Agent def.	Markdown files	Code (DAG)	YAML/Code	Code (class)
Triển khai	Self-hosted	Tích hợp app	Tích hợp app	Tích hợp app
Độ phức tạp	Thấp (config)	Cao (code)	Thấp-TB	Trung bình
GitHub stars	**~350K**	~12K	~25K	~40K

7. Năng lực nổi bật

7.1. Model-agnostic (BYOM)

Hơn 20 providers, mỗi agent dùng model khác nhau. Tiết kiệm 35-50% token.

7.2. Multi-channel

25+ nền tảng nhắn tin, chuẩn hóa qua Channel Adapters.

7.3. Local-first

Dữ liệu lưu local (JSONL, Markdown). Không cloud, không database.

7.4. Multi-agent 3 tầng

Bindings + Sub-agents + Lobster, xâu chuỗi được trong cùng một tác vụ.

7.5. Cấu hình Markdown

SOUL.md, AGENTS.md, TOOLS.md thay vì code. Rào cản gia nhập thấp.

7.6. Hệ sinh thái

350K+ stars, 1.000+ skills (ClawHub), DigitalOcean 1-click, NVIDIA NemoClaw, ClawFlows.

8. Rủi ro và hạn chế

8.1. Bảo mật

- Quyền truy cập rộng, dễ bị prompt injection. CVE-2026-25253 đã ghi nhận.
- ClawHub: ~20% skills chứa mã độc. Trung Quốc hạn chế từ 3/2026.

8.2. Kiến trúc

- Workspace không phải hard sandbox. Sub-agent giới hạn depth 2, concurrent 8.
- Lobster cài phức tạp, gateway fail âm thầm nếu binary không trên PATH.

8.3. Chi phí

- Anthropic siết subscription Claude cho OpenClaw (4/2026).
- Cập nhật thường xuyên thay đổi permission model, gây gãy pipeline.

9. Tài liệu tham khảo

9.1. Chính thức

- GitHub: github.com/openclaw/openclaw
- Docs: docs.openclaw.ai (/concepts/multi-agent, /concepts/delegate-architecture)
- Blog: openclaw.ai/blog — “Introducing OpenClaw” (29/01/2026)
- Lobster: github.com/openclaw/lobster

9.2. Phân tích cộng đồng

- DEV.to: Gustavo Gondim — deterministic multi-agent dev pipeline
- Meta Intelligence: subagents, agent teams & orchestration
- LumaDock: multi-agent setup, coordination & governance

9.3. Repo cộng đồng

- mergisi/awesome-openclaw-agents (162+ templates)
- shenhao-stu/openclaw-agents (9 agents, 1-command setup)
- raulvidis/openclaw-multi-agent-kit (10 agents + Telegram)